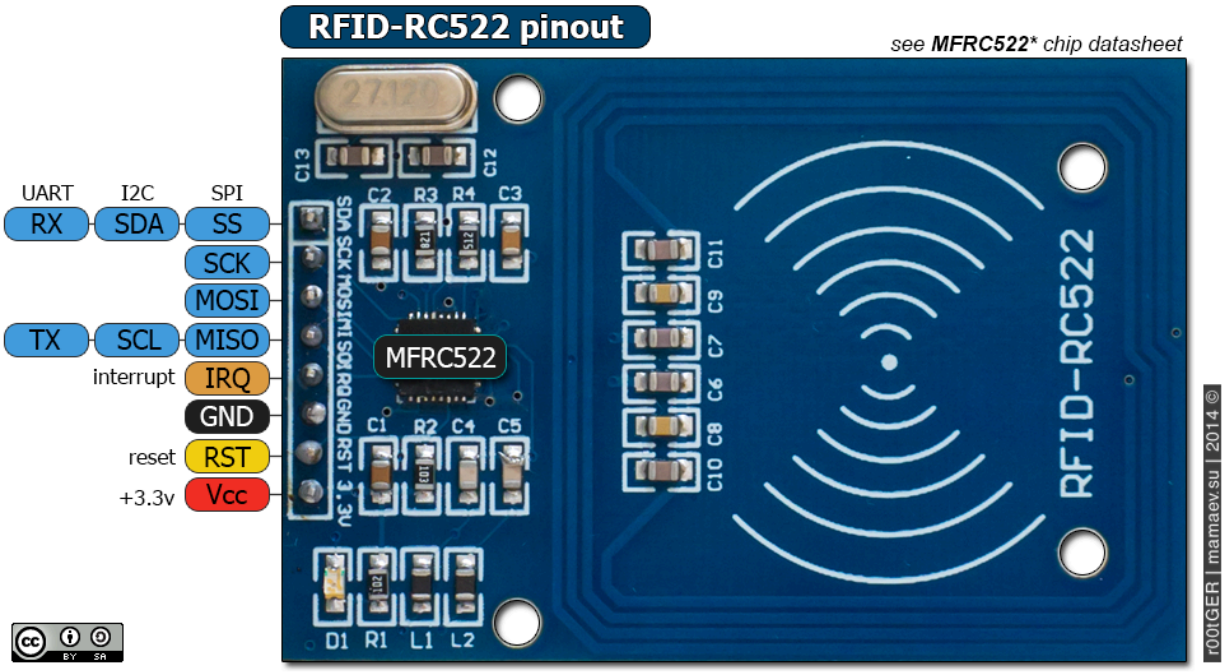
RFID - SPI

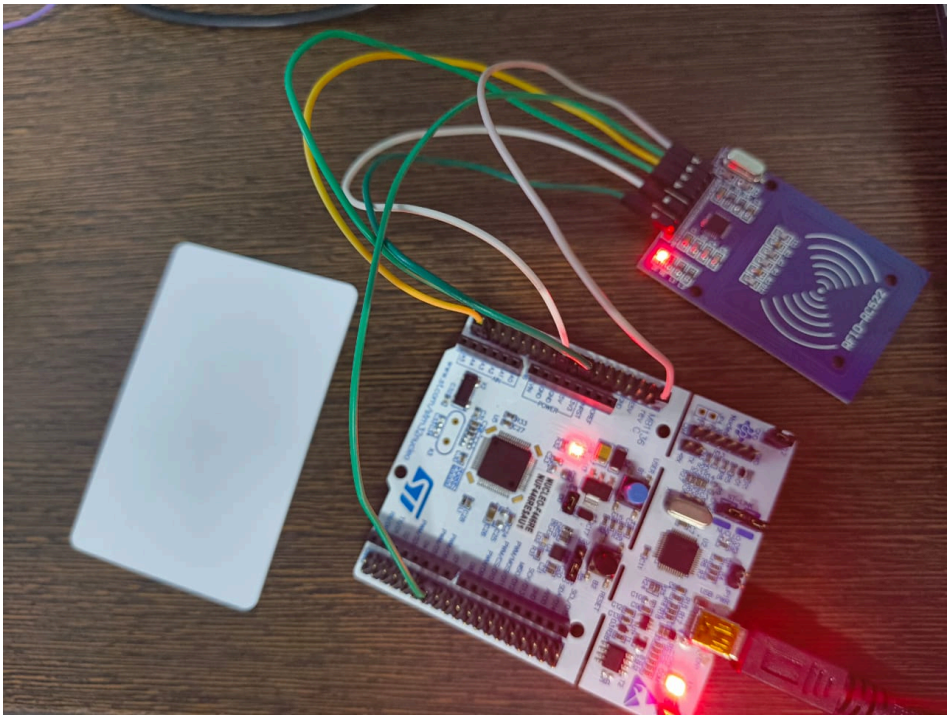


RFID SPI refers to using the Serial Peripheral Interface (SPI) protocol to communicate with an RFID reader module, most commonly the popular, low-cost MFRC522 (RC522) chip, to read/write 13.56MHz RFID tags (like MIFARE cards) with a microcontroller (Arduino, STM, etc.). SPI offers faster, simpler communication than I2C/UART, using pins like SCK, MISO, MOSI, and Chip Select (CS) for quick data exchange in projects like access control, attendance systems, and smart devices.

Why SPI?

- Speed: Faster data rates (up to 10Mbps) compared to I2C or UART for this application.
- Simplicity: Easier to implement for many hobbyists and developers.
- Reliability: Ensures stable data transfer for real-time applications like door locks or payment systems.

Connection:



STM32F446RE	RFID - RC522
PC1	MOSI
PC2	MISO
PB10	SCK
PC11	SDA
3.3v	VCC
GND	GND

Code:

```
/* main.c - RFID RC522 with STM32F446RE - Debug Version */
#include "main.h"
#include "string.h"
#include "stdio.h"
/* RC522 Register Definitions */
```

```
#define RC522_REG_COMMAND       0x01
#define RC522_REG_COMM_INT      0x04
#define RC522_REG_DIV_INT       0x05
#define RC522_REG_ERROR         0x06
#define RC522_REG_STATUS1       0x07
#define RC522_REG_STATUS2       0x08
#define RC522_REG_FIFO_DATA     0x09
#define RC522_REG_FIFO_LEVEL    0x0A
#define RC522_REG_CONTROL       0x0C
#define RC522_REG_BIT_FRAMING   0x0D
#define RC522_REG_MODE          0x11
#define RC522_REG_TX_CONTROL    0x14
#define RC522_REG_TX_ASK        0x15
#define RC522_REG_CRC_RESULT_L  0x21
#define RC522_REG_CRC_RESULT_H  0x22
#define RC522_REG_MOD_WIDTH     0x24
#define RC522_REG_RF_CFG        0x26
#define RC522_REG_TIMER_MODE    0x2A
#define RC522_REG_TIMER_PRESCALER 0x2B
#define RC522_REG_TIMER_RELOAD_H  0x2C
#define RC522_REG_TIMER_RELOAD_L  0x2D
#define RC522_REG_VERSION       0x37
/* RC522 Commands */
#define RC522_CMD_IDLE          0x00
#define RC522_CMD_MEM           0x01
#define RC522_CMD_CALC_CRC      0x03
#define RC522_CMD_TRANSMIT      0x04
#define RC522_CMD_RECEIVE       0x08
#define RC522_CMD_TRANSCEIVE    0x0C
#define RC522_CMD_AUTHENT       0x0E
#define RC522_CMD_SOFT_RESET    0x0F
/* PICC Commands */
#define PICC_CMD_REQA           0x26
#define PICC_CMD_WUPA           0x52
#define PICC_CMD_SEL_CL1        0x93
#define PICC_CMD_SEL_CL2        0x95
#define PICC_CMD_SEL_CL3        0x97
/* Status Codes */
#define STATUS_OK               0
#define STATUS_ERROR            1
#define STATUS_COLLISION        2
#define STATUS_TIMEOUT          3
#define STATUS_NO_ROOM          4
#define STATUS_INTERNAL_ERROR   5
/* Pin Definitions */
#define RC522_CS_PIN    GPIO_PIN_11
#define RC522_CS_PORT   GPIOC
#define RC522_RST_PIN   GPIO_PIN_12
#define RC522_RST_PORT  GPIOC
```

```c
/* Macros */
#define RC522_CS_LOW()   HAL_GPIO_WritePin(RC522_CS_PORT, RC522_CS_PIN,
GPIO_PIN_RESET)
#define RC522_CS_HIGH()  HAL_GPIO_WritePin(RC522_CS_PORT, RC522_CS_PIN,
GPIO_PIN_SET)
#define RC522_RST_LOW()  HAL_GPIO_WritePin(RC522_RST_PORT, RC522_RST_PIN,
GPIO_PIN_RESET)
#define RC522_RST_HIGH() HAL_GPIO_WritePin(RC522_RST_PORT, RC522_RST_PIN,
GPIO_PIN_SET)
/* Global Variables */
SPI_HandleTypeDef hspi2;  // Changed from hspi3 to hspi2
UART_HandleTypeDef huart2;
/* Function Prototypes */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_SPI2_Init(void);
static void MX_USART2_UART_Init(void);
uint8_t RC522_Init(void);
void RC522_WriteRegister(uint8_t addr, uint8_t val);
uint8_t RC522_ReadRegister(uint8_t addr);
void RC522_SetBitMask(uint8_t reg, uint8_t mask);
void RC522_ClearBitMask(uint8_t reg, uint8_t mask);
void RC522_AntennaOn(void);
uint8_t RC522_Request(uint8_t reqMode, uint8_t *TagType);
uint8_t RC522_ToCard(uint8_t command, uint8_t *sendData, uint8_t sendLen,
                     uint8_t *backData, uint16_t *backLen);
uint8_t RC522_Anticoll(uint8_t *serNum);
void RC522_Test(void);
void Debug_Print(char *msg) {
   HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 1000);
}
int main(void) {
   HAL_Init();
   SystemClock_Config();
   MX_GPIO_Init();
   MX_SPI2_Init();
   MX_USART2_UART_Init();
   uint8_t status;
   uint8_t tagType[2];
   uint8_t serialNum[5];
   char msg[128];
   Debug_Print("\r\n=== MFRC522 STM32F446RE RFID Reader ===\r\n");
   // Test SPI Communication
   RC522_Test();
   if (RC522_Init() != STATUS_OK) {
       Debug_Print("ERROR: RFID Init Failed!\r\n");
       Debug_Print("Check connections:\r\n");
       Debug_Print("  MOSI: PC1\r\n");
       Debug_Print("  MISO: PC2\r\n");
```

```
        Debug_Print("  SCK:  PB10\r\n");
        Debug_Print("  SDA:  PC11\r\n");
        //Debug_Print("  RST:  PC12\r\n");
        Debug_Print("  VCC:  3.3V\r\n");
        Debug_Print("  GND:  GND\r\n");
        while(1) {
            HAL_Delay(1000);
        }
    }
    Debug_Print("RFID Initialized Successfully\r\n\r\n");
    uint32_t lastCardTime = 0;
    uint8_t cardPresent = 0;
    while (1) {
        status = RC522_Request(PICC_CMD_REQA, tagType);
        if (status == STATUS_OK) {
            if (!cardPresent || (HAL_GetTick() - lastCardTime > 2000)) {
                sprintf(msg, "Card Detected! Type: 0x%02X%02X\r\n", tagType[0],
tagType[1]);
                Debug_Print(msg);
                status = RC522_Anticoll(serialNum);
                if (status == STATUS_OK) {
                    // Verify UID is not all 0x00 or 0xFF
                    uint8_t valid = 0;
                    uint8_t allZero = 1;
                    uint8_t allFF = 1;
                    for (int i = 0; i < 5; i++) {
                        if (serialNum[i] != 0x00) allZero = 0;
                        if (serialNum[i] != 0xFF) allFF = 0;
                    }
                    if (!allZero && !allFF) {
                        sprintf(msg, "UID: %02X:%02X:%02X:%02X:%02X\r\n\r\n",
                                serialNum[0], serialNum[1], serialNum[2],
                                serialNum[3], serialNum[4]);
                        Debug_Print(msg);
                        valid = 1;
                    } else {
                        Debug_Print("Invalid UID (all 00 or FF) - Communication
Error!\r\n\r\n");
                    }
                    if (valid) {
                        cardPresent = 1;
                        lastCardTime = HAL_GetTick();
                    }
                } else {
                    Debug_Print("Anticollision failed!\r\n\r\n");
                }
            }
        } else {
            if (cardPresent && (HAL_GetTick() - lastCardTime > 1000)) {
```

```
                cardPresent = 0;
            }
        }
        HAL_Delay(100);
    }
}
void RC522_Test(void) {
    char msg[100];
    Debug_Print("\r\n--- Testing SPI Communication ---\r\n");
    // Test 1: Read Version Register
    uint8_t version = RC522_ReadRegister(RC522_REG_VERSION);
    sprintf(msg, "Version Register: 0x%02X ", version);
    Debug_Print(msg);
    if (version == 0x91 || version == 0x92) {
        Debug_Print("(OK - Valid MFRC522)\r\n");
    } else if (version == 0x00 || version == 0xFF) {
        Debug_Print("(ERROR - No Communication!)\r\n");
        Debug_Print("  * Check wiring\r\n");
        Debug_Print("  * Verify 3.3V power\r\n");
        Debug_Print("  * Check CS pin is working\r\n");
    } else {
        sprintf(msg, "(Unknown version - possible clone)\r\n");
        Debug_Print(msg);
    }
    // Test 2: Write and Read Test
    Debug_Print("Write/Read Test: ");
    RC522_WriteRegister(RC522_REG_MOD_WIDTH, 0x25);
    HAL_Delay(1);
    uint8_t readback = RC522_ReadRegister(RC522_REG_MOD_WIDTH);
    sprintf(msg, "Wrote 0x25, Read 0x%02X ", readback);
    Debug_Print(msg);
    if (readback == 0x25) {
        Debug_Print("(OK)\r\n");
    } else {
        Debug_Print("(ERROR)\r\n");
    }
    Debug_Print("--- Test Complete ---\r\n\r\n");
}
uint8_t RC522_Init(void) {
    char msg[64];
    // Hardware reset
    RC522_CS_HIGH();
    RC522_RST_LOW();
    HAL_Delay(10);
    RC522_RST_HIGH();
    HAL_Delay(50);
    // Soft reset
    RC522_WriteRegister(RC522_REG_COMMAND, RC522_CMD_SOFT_RESET);
    HAL_Delay(50);
```

```c
    // Check if module is responding
    uint8_t version = RC522_ReadRegister(RC522_REG_VERSION);
    sprintf(msg, "Firmware Version: 0x%02X\r\n", version);
    Debug_Print(msg);
    if (version == 0x00 || version == 0xFF) {
        return STATUS_ERROR;
    }
    // Timer configuration
    RC522_WriteRegister(RC522_REG_TIMER_MODE, 0x8D);
    RC522_WriteRegister(RC522_REG_TIMER_PRESCALER, 0x3E);
    RC522_WriteRegister(RC522_REG_TIMER_RELOAD_L, 30);
    RC522_WriteRegister(RC522_REG_TIMER_RELOAD_H, 0);
    // TX configuration
    RC522_WriteRegister(RC522_REG_TX_ASK, 0x40);
    RC522_WriteRegister(RC522_REG_MODE, 0x3D);
    // Set RF gain to maximum
    RC522_WriteRegister(RC522_REG_RF_CFG, 0x70);
    // Turn on antenna
    RC522_AntennaOn();
    return STATUS_OK;
}
void RC522_WriteRegister(uint8_t addr, uint8_t val) {
    RC522_CS_LOW();
    HAL_Delay(1); // Small delay for stability
    uint8_t txData = (addr << 1) & 0x7E;
    HAL_SPI_Transmit(&hspi2, &txData, 1, 100);  // Changed to hspi2
    HAL_SPI_Transmit(&hspi2, &val, 1, 100);      // Changed to hspi2
    RC522_CS_HIGH();
    HAL_Delay(1);
}
uint8_t RC522_ReadRegister(uint8_t addr) {
    uint8_t val;
    RC522_CS_LOW();
    HAL_Delay(1);
    uint8_t txData = ((addr << 1) & 0x7E) | 0x80;
    HAL_SPI_Transmit(&hspi2, &txData, 1, 100);  // Changed to hspi2
    HAL_SPI_Receive(&hspi2, &val, 1, 100);      // Changed to hspi2
    RC522_CS_HIGH();
    HAL_Delay(1);
    return val;
}
void RC522_SetBitMask(uint8_t reg, uint8_t mask) {
    uint8_t tmp = RC522_ReadRegister(reg);
    RC522_WriteRegister(reg, tmp | mask);
}
void RC522_ClearBitMask(uint8_t reg, uint8_t mask) {
    uint8_t tmp = RC522_ReadRegister(reg);
    RC522_WriteRegister(reg, tmp & (~mask));
}
```

```c
void RC522_AntennaOn(void) {
   uint8_t temp = RC522_ReadRegister(RC522_REG_TX_CONTROL);
   if (!(temp & 0x03)) {
       RC522_SetBitMask(RC522_REG_TX_CONTROL, 0x03);
   }
}
uint8_t RC522_Request(uint8_t reqMode, uint8_t *TagType) {
   uint8_t status;
   uint16_t backBits;
   RC522_WriteRegister(RC522_REG_BIT_FRAMING, 0x07);
   TagType[0] = reqMode;
   status = RC522_ToCard(RC522_CMD_TRANSCEIVE, TagType, 1, TagType, &backBits);
   if ((status != STATUS_OK) || (backBits != 0x10)) {
       status = STATUS_ERROR;
   }
   return status;
}
uint8_t RC522_ToCard(uint8_t command, uint8_t *sendData, uint8_t sendLen,
                     uint8_t *backData, uint16_t *backLen) {
   uint8_t status = STATUS_ERROR;
   uint8_t irqEn = 0x77;
   uint8_t waitIRq = 0x30;
   uint8_t n;
   uint16_t i;
   if (command == RC522_CMD_AUTHENT) {
       irqEn = 0x12;
       waitIRq = 0x10;
   }
   RC522_WriteRegister(RC522_REG_COMM_INT, irqEn | 0x80);
   RC522_ClearBitMask(RC522_REG_COMM_INT, 0x80);
   RC522_SetBitMask(RC522_REG_FIFO_LEVEL, 0x80);
   RC522_WriteRegister(RC522_REG_COMMAND, RC522_CMD_IDLE);
   for (i = 0; i < sendLen; i++) {
       RC522_WriteRegister(RC522_REG_FIFO_DATA, sendData[i]);
   }
   RC522_WriteRegister(RC522_REG_COMMAND, command);
   if (command == RC522_CMD_TRANSCEIVE) {
       RC522_SetBitMask(RC522_REG_BIT_FRAMING, 0x80);
   }
   i = 2000;
   do {
       n = RC522_ReadRegister(RC522_REG_COMM_INT);
       i--;
   } while ((i != 0) && !(n & 0x01) && !(n & waitIRq));
   RC522_ClearBitMask(RC522_REG_BIT_FRAMING, 0x80);
   if (i != 0) {
       if (!(RC522_ReadRegister(RC522_REG_ERROR) & 0x1B)) {
           status = STATUS_OK;
           if (n & irqEn & 0x01) {
```

```
                status = STATUS_TIMEOUT;
            }
            if (command == RC522_CMD_TRANSCEIVE) {
                n = RC522_ReadRegister(RC522_REG_FIFO_LEVEL);
                uint8_t lastBits = RC522_ReadRegister(RC522_REG_CONTROL) & 0x07;
                if (lastBits) {
                    *backLen = (n - 1) * 8 + lastBits;
                } else {
                    *backLen = n * 8;
                }
                if (n == 0) {
                    n = 1;
                }
                if (n > 16) {
                    n = 16;
                }
                for (i = 0; i < n; i++) {
                    backData[i] = RC522_ReadRegister(RC522_REG_FIFO_DATA);
                }
            }
        } else {
            status = STATUS_ERROR;
        }
    }
    return status;
}
uint8_t RC522_Anticoll(uint8_t *serNum) {
    uint8_t status;
    uint8_t i;
    uint8_t serNumCheck = 0;
    uint16_t unLen;
    RC522_WriteRegister(RC522_REG_BIT_FRAMING, 0x00);
    serNum[0] = PICC_CMD_SEL_CL1;
    serNum[1] = 0x20;
    status = RC522_ToCard(RC522_CMD_TRANSCEIVE, serNum, 2, serNum, &unLen);
    if (status == STATUS_OK) {
        for (i = 0; i < 4; i++) {
            serNumCheck ^= serNum[i];
        }
        if (serNumCheck != serNum[i]) {
            status = STATUS_ERROR;
        }
    }
    return status;
}
/* SPI2 Init - SLOWER speed for better reliability */
static void MX_SPI2_Init(void) {
    hspi2.Instance = SPI2;
    hspi2.Init.Mode = SPI_MODE_MASTER;
```

```
   hspi2.Init.Direction = SPI_DIRECTION_2LINES;
   hspi2.Init.DataSize = SPI_DATASIZE_8BIT;
   hspi2.Init.CLKPolarity = SPI_POLARITY_LOW;
   hspi2.Init.CLKPhase = SPI_PHASE_1EDGE;
   hspi2.Init.NSS = SPI_NSS_SOFT;
   hspi2.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_64; // Slower for
reliability
   hspi2.Init.FirstBit = SPI_FIRSTBIT_MSB;
   hspi2.Init.TIMode = SPI_TIMODE_DISABLE;
   hspi2.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
   hspi2.Init.CRCPolynomial = 10;
   if (HAL_SPI_Init(&hspi2) != HAL_OK) {
       Error_Handler();
   }
}
static void MX_GPIO_Init(void) {
   GPIO_InitTypeDef GPIO_InitStruct = {0};
   __HAL_RCC_GPIOC_CLK_ENABLE();
   /* CS and RST pins */
   HAL_GPIO_WritePin(GPIOC, RC522_CS_PIN | RC522_RST_PIN, GPIO_PIN_SET);
   GPIO_InitStruct.Pin = RC522_CS_PIN | RC522_RST_PIN;
   GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
   GPIO_InitStruct.Pull = GPIO_NOPULL;
   GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
   HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
}
static void MX_USART2_UART_Init(void) {
   huart2.Instance = USART2;
   huart2.Init.BaudRate = 115200;
   huart2.Init.WordLength = UART_WORDLENGTH_8B;
   huart2.Init.StopBits = UART_STOPBITS_1;
   huart2.Init.Parity = UART_PARITY_NONE;
   huart2.Init.Mode = UART_MODE_TX_RX;
   huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
   huart2.Init.OverSampling = UART_OVERSAMPLING_16;
   if (HAL_UART_Init(&huart2) != HAL_OK) {
       Error_Handler();
   }
}
/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
 RCC_OscInitTypeDef RCC_OscInitStruct = {0};
 RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
 /** Configure the main internal regulator output voltage
 */
```

```
  __HAL_RCC_PWR_CLK_ENABLE();
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);
  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLM = 16;
  RCC_OscInitStruct.PLL.PLLN = 336;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV4;
  RCC_OscInitStruct.PLL.PLLQ = 2;
  RCC_OscInitStruct.PLL.PLLR = 2;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }
  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
}
void Error_Handler(void) {
    __disable_irq();
    while (1) {
    }
}
```

O/P:

COM16 - Tera Term VT

File Edit Setup Control Window Help

```
Firmware Version: 0xB2
RFID Initialized Successfully


=== MFRC522 STM32F446RE RFID Reader ===

--- Testing SPI Communication ---
Version Register: 0xB2 (Unknown version - possible clone)
Write/Read Test: Wrote 0x25, Read 0x25 (OK)
--- Test Complete ---

Firmware Version: 0xB2
RFID Initialized Successfully

Card Detected! Type: 0x0400
UID: 43:01:72:15:25

Card Detected! Type: 0x0400
UID: 43:01:72:15:25

Card Detected! Type: 0x0400
UID: 43:01:72:15:25

Card Detected! Type: 0x0400
```